

ESP32 Bluetooth RC Controller

Main Entry Point Documentation

Miguel Chip

February 12, 2026

Contents

1	Project Entry File: ESP32_BT_Controller.ino	4
1.1	Purpose	4
1.2	Módulos incluidos	4
1.3	Setup Function	5
1.4	Bucle Principal de Ejecución	5
2	Capa de Comunicación Bluetooth	7
2.1	Propósito	7
2.2	Estrategia de Diseño	7
2.3	Archivo de Cabecera: bluetooth.h	7
2.4	Archivo de Implementación: bluetooth.cpp	7
2.5	Acceso Global al Objeto	8
2.6	Descubrimiento del Dispositivo	8
2.7	Por Qué Esta Arquitectura Funciona	8
2.8	Flujo de Datos del Sistema	8
3	Capa de Definición de Paquetes	9
3.1	Propósito	9
3.2	Reglas de Diseño	9
3.3	Archivo de Cabecera: packets.h	9
3.3.1	Paquete de Estado RC	9
3.3.2	Contenedor Union	10
3.3.3	Paquete de Evento	10
3.4	Paquetes de Telemetría	10
3.4.1	PanelPacket	10
3.4.2	IndicatorPacket	11
3.4.3	PlotPacket	11
3.5	Paquete de Telemetría de Entradas	11
3.6	Paquete de Sensores I2C	11
3.7	Instancias Globales de Paquetes	12
3.8	Constantes de Tamaño de Paquetes	12

4	Capa de Recepción Bluetooth	12
4.1	Propósito	12
4.2	Responsabilidades	12
4.3	Archivo de Cabecera: <code>receiver.h</code>	13
4.4	Archivo Fuente: <code>receiver.cpp</code>	13
4.4.1	Inclusiones	13
4.4.2	Buffer de Recepción	14
4.4.3	Función Principal: <code>handleBluetooth()</code>	14
4.4.4	Lectura de Bytes Entrantes	14
4.4.5	Búsqueda de Paquetes	15
4.4.6	Extracción del Paquete	15
4.4.7	Disparo de Evento	15
4.4.8	Limpieza del Buffer	15
4.5	Por Qué un Buffer Circular es Crítico	16
5	Capa de Transmisión de Telemetría	16
5.1	Propósito	16
5.2	Canales de Paquetes	16
5.3	Archivo de Cabecera: <code>telemetry.h</code>	17
5.4	Archivo Fuente: <code>telemetry.cpp</code>	17
5.4.1	Inclusiones	17
5.4.2	Estado de Temporización	17
5.4.3	Intervalos de Transmisión	17
5.4.4	Metadatos de Gráficas	18
5.5	Paquete de Configuración	18
5.6	Emisor Periódico de Telemetría	18
5.6.1	Telemetría de Panel	18
5.6.2	Telemetría de Indicadores	19
5.6.3	Telemetría de Gráficas	19
5.7	Modelo de Checksum	19
5.8	Rol en el Sistema	19
6	Capa de Control (Subsistema de Salida de Hardware)	20
6.1	Propósito	20
6.2	Archivo de cabecera: <code>control.h</code>	20
6.3	Arquitectura de Hardware	20
6.3.1	Categorías de señales	20
6.4	Mapa de pines	21
6.5	Configuración PWM	21
6.6	Función de inicialización	22
6.7	Bucle principal de control	22
6.7.1	Maapeo de sticks	22
6.7.2	Maapeo de servo	22
6.8	Maapeo de knobs	22
6.9	Decodificación de switches	22
6.10	Manejo de eventos	23
6.11	Rol del sistema	23

7	Capa de Entrada (Telemetría Local de Sensores y Botones)	23
7.1	Propósito	23
7.2	Archivo de cabecera: input.h	23
7.3	Entradas soportadas	24
7.3.1	Entradas analógicas	24
7.3.2	Entradas digitales	24
7.4	Formato del paquete	24
7.5	Control de tiempo	25
7.6	Inicialización	25
7.7	Bucle principal de actualización	25
7.8	Adquisición de datos	25
7.9	Generación del checksum	26
7.10	Transmisión	26
7.11	Integración del sistema	26
8	Capa de Telemetría de Sensores I2C	26
8.1	Propósito	26
8.2	Interfaz pública: i2c_sensors.h	27
8.3	Configuración del bus I2C	27
8.4	Sensores soportados	27
8.4.1	Sensor de temperatura	27
8.4.2	IMU (acelerómetro)	28
8.5	Paquete de telemetría I2C	28
8.6	Frecuencia de transmisión	28
8.7	Emisor de telemetría	28
8.8	Construcción del paquete	28
8.9	Checksum y transmisión	29
8.10	Integración del sistema	29
9	Debug and Diagnostic Layer	29
9.1	Purpose	29
9.2	Header File: debug.h	30
9.3	Implementation File: debug.cpp	30
9.3.1	State Packet Printer	30
9.3.2	Event Packet Printer	31
9.3.3	Panel Telemetry Printer	31
9.3.4	Indicator Telemetry Printer	31
9.3.5	Plot Telemetry Printer	31
9.4	Why a Dedicated Debug Layer?	32

1 Project Entry File: ESP32_BT_Controller.ino

1.1 Purpose

Este archivo es el **punto de entrada principal** del sistema RC Bluetooth. No controla directamente motores o sensores. Solo **coordina todos los módulos**:

- Comunicación Bluetooth
- Decodificación de paquetes
- Telemetría
- Control de hardware
- Entradas locales y sensores I2C

Este diseño se llama **arquitectura modular por capas**. Cada módulo es independiente y más fácil de mantener.

1.2 Módulos incluidos

```
#include <Arduino.h>
#include "bluetooth.h"
#include "packets.h"
#include "telemetry.h"
#include "receiver.h"
#include "debug.h"
#include "control.h"
#include "input.h"
#include "i2c_sensors.h"
```

Descripción de componentes

File	Purpose
Arduino.h	Core ESP32 and Arduino functions
bluetooth.h	Bluetooth SPP and SerialBT setup
packets.h	RC data packet structures
telemetry.h	Telemetry transmission logic
receiver.h	Bluetooth RX and packet decoding
debug.h	Serial debugging helpers
control.h	Hardware outputs (PWM, motors, LEDs)
input.h	Local ESP32 inputs
i2c_sensors.h	Sensor bus over I2C

1.3 Setup Function

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);  
    Serial.println("Booting...");  
    SerialBT.begin(DEVICE_NAME);  
    Serial.println("ESP32 Bluetooth Receiver Ready.");  
    Serial.printf(  
        "Listening for State (%d) and Event (%d)\n",  
        STATE_PACKET_SIZE, EVENT_PACKET_SIZE  
    );  
    controlInit();  
    i2cInit();  
}
```

Resumen operativo

- Inicia la consola USB a 115200 baudios.
- Inicia Bluetooth SPP.
- Muestra mensajes de diagnóstico.
- Inicializa todas las salidas.
- Inicializa sensores I2C.

1.4 Bucle Principal de Ejecución

Motor de Control en Tiempo Real

La función `loop()` es el motor de ejecución en tiempo real del controlador RC con ESP32. Se ejecuta de forma continua después de que finaliza `setup()`, y puede ejecutarse miles de veces por segundo.

Esta función sincroniza:

- Datos de control recibidos desde la aplicación Android
- Salidas físicas (motores, servos, luces, buzzer)
- Telemetría enviada de regreso a Android
- Entradas locales conectadas al ESP32
- Sensores I2C como voltaje, corriente o temperatura

Cada tarea está aislada en su propia función para respetar el principio de **separación de responsabilidades**. Ninguna función bloquea el sistema usando retardos (`delay`), por lo que el controlador se mantiene siempre responsivo.

Código Fuente

```
void loop() {  
    handleBluetooth();  
    sendTelemetryIfDue();  
    controlUpdate();  
    inputUpdate();  
    sendI2CTelemetry();  
}
```

Descripción Línea por Línea

1. `handleBluetooth();`

Lee los bytes entrantes desde el enlace Bluetooth SPP. Esta función reconstruye los paquetes enviados por la app Android, valida su checksum, identifica si son paquetes de estado o de evento, y actualiza las estructuras globales definidas en `packets.h`.

2. `sendTelemetryIfDue();`

Envía paquetes de telemetría a la aplicación Android solo cuando ha transcurrido el intervalo de tiempo definido. Esto evita saturar el canal Bluetooth sin perder fluidez en la interfaz.

3. `controlUpdate();`

Convierte las posiciones de los sticks, valores de perillas y estados de interruptores en salidas físicas reales. Actualiza los canales PWM para motores y servos, y configura salidas digitales para luces, relés y dispositivos auxiliares.

4. `inputUpdate();`

Lee entradas locales como botones físicos, potenciómetros o sensores digitales conectados al ESP32. Cuando se detecta un cambio, se envía un evento a la aplicación Android.

5. `sendI2CTelemetry();`

Lee todos los sensores I2C conectados y empaqueta sus valores en tramas de telemetría que se transmiten a la aplicación Android.

Justificación del Orden de Ejecución

1. Recibir primero los datos de control
2. Transmitir la telemetría
3. Actualizar las salidas de hardware
4. Leer las entradas locales
5. Enviar telemetría de sensores

Este orden garantiza un comportamiento determinista, una respuesta rápida y un control seguro de los motores.

2 Capa de Comunicación Bluetooth

2.1 Propósito

La capa Bluetooth proporciona el enlace de comunicación inalámbrica entre el controlador ESP32 y la aplicación Android utilizando el **Serial Port Profile (SPP)**.

Este módulo:

- Crea una interfaz serial Bluetooth global
- Define el nombre del dispositivo Bluetooth
- Hace que el objeto Bluetooth sea accesible desde todos los módulos

No contiene lógica ni procesamiento de datos. Solo proporciona el canal de comunicación.

2.2 Estrategia de Diseño

En lugar de crear múltiples objetos Bluetooth, este sistema utiliza una **única instancia global de BluetoothSerial** compartida entre todos los módulos.

Esto se logra usando la palabra clave `extern` en el archivo de cabecera y una única definición en el archivo de implementación.

2.3 Archivo de Cabecera: `bluetooth.h`

```
#ifndef BLUETOOTH_H
#define BLUETOOTH_H

#include <BluetoothSerial.h>

extern BluetoothSerial SerialBT;

#define DEVICE_NAME "ESP32-BT"

#endif
```

2.4 Archivo de Implementación: `bluetooth.cpp`

```
#include "bluetooth.h"

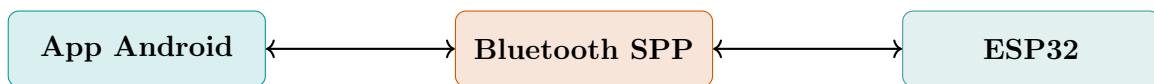
BluetoothSerial SerialBT;
```

2.5 Acceso Global al Objeto

Cualquier módulo que incluya `bluetooth.h` puede utilizar:

```
SerialBT.begin (DEVICE_NAME) ;  
SerialBT.available () ;  
SerialBT.read () ;  
SerialBT.write (... ) ;  
SerialBT.hasClient () ;
```

Esto permite una comunicación transparente entre:



2.6 Descubrimiento del Dispositivo

El ESP32 se anuncia con el nombre:

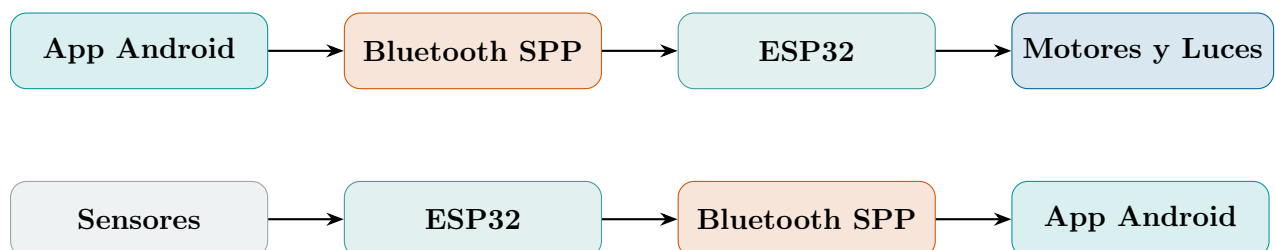
`ESP32-BT`

Este nombre aparece en la lista de dispositivos Bluetooth de Android e identifica a tu controlador RC.

2.7 Por Qué Esta Arquitectura Funciona

- Evita múltiples instancias de la pila Bluetooth
- Reduce el consumo de memoria
- Permite un diseño modular limpio
- Mantiene Bluetooth independiente de las capas de lógica

2.8 Flujo de Datos del Sistema



3 Capa de Definición de Paquetes

3.1 Propósito

La capa de paquetes define cada mensaje binario que se intercambia entre la aplicación Android y el controlador ESP32.

Esta capa es la **única fuente de verdad** para todas las estructuras y tamaños de comunicación. Todos los demás módulos (`receiver.cpp`, `telemetry.cpp`, `control.cpp`, etc.) dependen de los tipos de datos declarados aquí.

El sistema de paquetes está diseñado con los siguientes objetivos:

- Tramas de tamaño fijo
- Bajo consumo de ancho de banda para Bluetooth SPP
- Análisis determinista de datos
- Uso mínimo de memoria RAM
- Verificación rápida de checksum

3.2 Reglas de Diseño

Todos los paquetes en este sistema siguen las mismas reglas de protocolo:

- Cada paquete comienza con dos bytes de cabecera
- Cada paquete termina con un byte de verificación (checksum)
- Todas las estructuras están marcadas como `packed` para evitar relleno
- Los *unions* permiten acceso como bytes crudos o como estructura
- Todos los paquetes están definidos en un único archivo de cabecera

3.3 Archivo de Cabecera: `packets.h`

3.3.1 Paquete de Estado RC

Este es el paquete principal de control enviado continuamente por la aplicación Android.

```
typedef struct __attribute__((packed)) {  
    byte startByte1, startByte2;  
    uint16_t leftStickX, leftStickY;  
    uint16_t rightStickX, rightStickY;  
    uint16_t leftKnob, rightKnob;  
    byte switches;  
    byte checksum;  
    byte endByte1, endByte2;  
} RcPacket;
```

Campos

- `startByte1`, `startByte2` — cabecera fija (sincronización)
- `leftStickX`, `leftStickY` — dirección y aceleración
- `rightStickX`, `rightStickY` — paneo auxiliar y segundo motor
- `leftKnob`, `rightKnob` — controles analógicos para luces y buzzer
- `switches` — interruptores digitales empaquetados en bits
- `checksum` — verificación aditiva
- `endByte1`, `endByte2` — delimitadores de trama

3.3.2 Contenedor Union

```
typedef union {  
    RcPacket data;  
    byte bytes[sizeof(RcPacket)];  
} StatePacketUnion;
```

Esto permite:

- `rcStatePacket.data.leftStickX`
- `rcStatePacket.bytes[i]`

3.3.3 Paquete de Evento

Los paquetes de evento solo se envían cuando se presiona un botón.

```
typedef struct __attribute__((packed)) {  
    byte startByte1, startByte2;  
    byte eventId;  
    byte checksum;  
} EventPacket;
```

3.4 Paquetes de Telemetría

3.4.1 PanelPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    uint16_t leftPanelValue, rightPanelValue;  
    byte panelStates;  
    byte checksum;  
} PanelPacket;
```

Se utiliza para transmitir valores de indicadores del panel.

3.4.2 IndicatorPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    byte analogValue, batteryLevel;  
    byte checksum;  
} IndicatorPacket;
```

Se utiliza para pequeños indicadores visuales.

3.4.3 PlotPacket

```
typedef struct __attribute__((packed)) {  
    byte header1, header2;  
    byte numPlots;  
    byte plotValue1, plotValue2, plotValue3;  
    byte checksum;  
} PlotPacket;
```

Se utiliza para gráficas de señales en tiempo real.

3.5 Paquete de Telemetría de Entradas

Este paquete reporta las entradas físicas conectadas directamente al ESP32.

```
typedef struct {  
    byte h1;  
    byte h2;  
    uint16_t a34;  
    uint16_t a35;  
    uint16_t a36;  
    uint16_t a39;  
    byte d0;  
    byte d16;  
    byte d17;  
    byte checksum;  
} InputPacket;
```

Los canales analógicos provienen de GPIO34, 35, 36 y 39. Las entradas digitales provienen de GPIO0, 16 y 17.

3.6 Paquete de Sensores I2C

```
typedef struct {  
    byte h1;  
    byte h2;  
    uint16_t temp;  
    uint16_t ax;  
    uint16_t ay;
```

```

    uint16_t az;
    byte sensorFlags;
    byte checksum;
} I2CPacket;

```

Se utiliza para sensores como sondas de temperatura e IMUs.

3.7 Instancias Globales de Paquetes

Como los archivos de cabecera no deben reservar memoria, todos los paquetes globales se definen dentro de `packets.cpp`.

```

StatePacketUnion rcStatePacket;
EventPacketUnion rcEventPacket;
PanelPacket panelPacket;
IndicatorPacket indicatorPacket;
PlotPacket plotPacket;
InputPacket inputPacket;
I2CPacket i2cPacket;

```

3.8 Constantes de Tamaño de Paquetes

Cada paquete tiene un tamaño constante exportado para validación y análisis:

```

const int STATE_PACKET_SIZE;
const int EVENT_PACKET_SIZE;
const int PANEL_PACKET_SIZE;
const int INDICATOR_PACKET_SIZE;
const int PLOT_PACKET_SIZE;

```

4 Capa de Recepción Bluetooth

4.1 Propósito

La Capa de Recepción Bluetooth es responsable de convertir un flujo continuo y no estructurado de bytes Bluetooth en paquetes de control válidos.

Este módulo es la **puerta de entrada** del sistema. Cada comando proveniente de la aplicación Android debe pasar por esta capa antes de poder afectar motores, servos, LEDs o telemetría.

4.2 Responsabilidades

El subsistema receptor realiza las siguientes tareas:

- Leer bytes crudos desde Bluetooth SPP

- Almacenarlos en un buffer circular
- Buscar cabeceras de paquetes válidas
- Extraer paquetes completos
- Copiar los bytes en *unions* estructurados
- Llamar a funciones de depuración visual
- Disparar eventos de control

4.3 Archivo de Cabecera: `receiver.h`

```
#ifndef RECEIVER_H
#define RECEIVER_H
void handleBluetooth();
#endif
```

Descripción

- Declara una única función pública: `handleBluetooth()`
- Esta función se llama continuamente desde el `loop()`
- No devuelve datos: actualiza estructuras globales de paquetes

4.4 Archivo Fuente: `receiver.cpp`

4.4.1 Inclusiones

```
#include <Arduino.h>
#include "bluetooth.h"
#include "packets.h"
#include "receiver.h"
#include "debug.h"
#include "control.h"
```

Estos archivos proporcionan:

- El flujo Bluetooth (`SerialBT`)
- Definiciones de paquetes
- Funciones de depuración
- Enlaces con la capa de control

4.4.2 Buffer de Recepción

```
#define RX_BUFFER_SIZE 64
```

Este buffer es intencionalmente más grande que cualquier paquete. Permite al sistema sobrevivir a:

- Paquetes fragmentados
- Múltiples paquetes en una sola ráfaga
- Ruido o flujos desalineados

4.4.3 Función Principal: handleBluetooth()

```
void handleBluetooth() {  
    static byte buffer[RX_BUFFER_SIZE];  
    static int bytesRead = 0;  
}
```

¿Por qué static?

- El buffer debe conservar su contenido entre iteraciones del `loop()`
- Los paquetes Bluetooth pueden llegar lentamente o por partes

4.4.4 Lectura de Bytes Entrantes

```
while (SerialBT.available()) {  
    if (bytesRead < RX_BUFFER_SIZE) {  
        buffer[bytesRead++] = SerialBT.read();  
    } else {  
        memmove(buffer, buffer + 1, RX_BUFFER_SIZE - 1);  
        buffer[RX_BUFFER_SIZE - 1] = SerialBT.read();  
    }  
}
```

Esta lógica:

- Agrega nuevos bytes hasta llenar el buffer
- Si está lleno, descarta el byte más antiguo
- Siempre conserva los datos más recientes

4.4.5 Búsqueda de Paquetes

Detección de Paquete de Estado

```
for (int i = 0; i <= bytesRead - 2; i++) {
    if (buffer[i] == 0xAA &&
        buffer[i + 1] == 0x55) {
        if (bytesRead - i >= STATE_PACKET_SIZE) {
            packetStart = i;
            packetType = 1;
            break;
        }
    }
}
```

Este bucle escanea el buffer buscando cabeceras válidas.

Detección de Paquete de Evento

```
if (buffer[i] == 0xBB &&
    buffer[i + 1] == 0x66) {
    if (bytesRead - i >= EVENT_PACKET_SIZE) {
        packetStart = i;
        packetType = 2;
        break;
    }
}
```

4.4.6 Extracción del Paquete

```
memcpy(rcStatePacket.bytes, &buffer[packetStart], STATE_PACKET_SIZE);
```

Esto copia los bytes crudos al *union*, haciendo que los campos estructurados estén disponibles de inmediato.

4.4.7 Disparo de Evento

```
controlHandleEvent(rcEventPacket.data.eventId);
```

Esto genera inmediatamente la salida de hardware definida en la capa de control.

4.4.8 Limpieza del Buffer

```
memmove(buffer, buffer + removeCount, bytesRead - removeCount);
bytesRead -= removeCount;
```

Esto elimina el paquete procesado y desplaza los bytes restantes.

4.5 Por Qué un Buffer Circular es Crítico

Sin un buffer circular, la fragmentación Bluetooth corrompería los paquetes. Este diseño garantiza:

- Ninguna pérdida de bytes
- Sin desincronización
- Control continuo en tiempo real

5 Capa de Transmisión de Telemetría

5.1 Propósito

El módulo de telemetría es responsable de enviar datos estructurados del sistema desde el ESP32 hacia la aplicación Android a través de Bluetooth.

A diferencia del módulo receptor, este subsistema es **solo de escritura**. Nunca interpreta bytes entrantes. Su única función es:

- Construir paquetes de telemetría
- Codificar valores del sistema en tiempo real
- Calcular sumas de verificación (checksums)
- Enviar flujos de bytes crudos
- Regular los tiempos de transmisión

5.2 Canales de Paquetes

Cada mensaje de telemetría comienza con el encabezado común 0xCC y un identificador de tipo:

Encabezado	Tipo	Descripción
0xCC	0x11	PanelPacket
0xCC	0x22	IndicatorPacket
0xCC	0x33	PlotPacket
0xCC	0x44	ConfigPacket (longitud variable)

5.3 Archivo de Cabecera: `telemetry.h`

```
#ifndef TELEMETRY_H
#define TELEMETRY_H
void sendTelemetryIfDue();
void sendConfigTelemetry();
#endif
```

Descripción

- `sendTelemetryIfDue()` envía paquetes periódicos
- `sendConfigTelemetry()` envía datos de configuración de longitud variable

5.4 Archivo Fuente: `telemetry.cpp`

5.4.1 Inclusiones

```
#include <Arduino.h>
#include <math.h>
#include "bluetooth.h"
#include "packets.h"
#include "telemetry.h"
```

5.4.2 Estado de Temporización

```
static unsigned long lastPanelSend = 0;
static unsigned long lastIndicatorSend = 0;
static unsigned long lastPlotSend = 0;
```

Cada variable almacena el instante de la última transmisión de cada tipo de paquete.

5.4.3 Intervalos de Transmisión

```
const long panelInterval = 1000;
const long indicatorInterval = 250;
const long plotInterval = 50;
```

- Panel: una vez por segundo
- Indicador: cuatro veces por segundo
- Gráfica: veinte veces por segundo

5.4.4 Metadatos de Gráficas

```
const char* plotNames[] = {"Volts", "Amps", "RPMs"};
const int NUM_PLOT_NAMES = 3;
```

Estas cadenas se envían a la interfaz Android para rotular las gráficas.

5.5 Paquete de Configuración

```
void sendConfigTelemetry() {
    byte buf[128]; int idx=0;
    buf[idx++]=0xCC; buf[idx++]=0x44;
    buf[idx++]=NUM_PLOT_NAMES;
    for(int i=0; i<NUM_PLOT_NAMES; i++) {
        byte l=strlen(plotNames[i]);
        buf[idx++]=l;
        memcpy(&buf[idx], plotNames[i], l);
        idx+=l;
    }
    if(SerialBT.hasClient()) SerialBT.write(buf, idx);
}
```

Descripción

- Encabezado: 0xCC 0x44
- Primer byte: número de nombres de gráficas
- Cada nombre se envía como: [longitud][caracteres]
- El tamaño del paquete es dinámico

5.6 Emisor Periódico de Telemetría

```
void sendTelemetryIfDue() {
    unsigned long t=millis();
    if(!SerialBT.hasClient()) return;
}
```

Detiene el envío si no hay ningún cliente Bluetooth conectado.

5.6.1 Telemetría de Panel

```
panelPacket={0xCC, 0x11, 1234, 5678, 0, 0};
byte c=0;
for(int i=2; i<PANEL_PACKET_SIZE-1; i++)
    c+=((byte*)&panelPacket)[i];
panelPacket.checksum=c;
SerialBT.write((byte*)&panelPacket, PANEL_PACKET_SIZE);
```

Campos

- Valor del panel izquierdo
- Valor del panel derecho
- Estados del panel
- Checksum aditivo

5.6.2 Telemetría de Indicadores

```
indicatorPacket={0xCC,0x22,75,98,0};
byte c=0;
for(int i=2;i<INDICATOR_PACKET_SIZE-1;i++)
c+=((byte*)&indicatorPacket)[i];
indicatorPacket.checksum=c;
SerialBT.write((byte*)&indicatorPacket,INDICATOR_PACKET_SIZE);
```

5.6.3 Telemetría de Gráficas

```
float s=t/1000.0f;
plotPacket={0xCC,0x33,3,
  (byte)((sin(s)*0.4)+0.6)*255,
  (byte)((cos(s)*0.2)+0.25)*255,
  (byte)((cos(s)*0.2)+0.5)*255,0};
```

Estos valores generan ondas en tiempo real para probar la interfaz.

5.7 Modelo de Checksum

Todos los paquetes de telemetría usan un checksum aditivo simple:

```
for(int i=2;i<PACKET_SIZE-1;i++)
checksum += rawBytes[i];
```

Esto verifica la integridad de los datos sin costo significativo de CPU.

5.8 Rol en el Sistema

La capa de telemetría proporciona a la app Android:

- Indicadores en tiempo real
- Estados del sistema
- Gráficas dinámicas
- Configuración de la interfaz

6 Capa de Control (Subsistema de Salida de Hardware)

6.1 Propósito

La capa de control es la interfaz física entre los datos RC digitales recibidos desde la aplicación Android y el hardware real conectado al ESP32.

Este módulo traduce:

- Posiciones de los sticks
- Valores de los knobs
- Estados de los switches
- Comandos de eventos

en señales eléctricas de control tales como:

- PWM para motores, servos, LEDs y buzzers
- Salidas digitales HIGH/LOW para relés, MOSFETs y módulos lógicos

La capa de control es completamente reactiva y sin estado: no almacena modos ni lógica. Solo aplica el estado RC actual al hardware.

6.2 Archivo de cabecera: `control.h`

```
#ifndef CONTROL_H
#define CONTROL_H

#include <Arduino.h>

// Inicializa todas las salidas de hardware
void controlInit();

// Se llama en cada loop para aplicar el estado RC al hardware
void controlUpdate();

// Se llama cuando se recibe un EventPacket
void controlHandleEvent(byte eventId);

#endif
```

6.3 Arquitectura de Hardware

6.3.1 Categorías de señales

- **Salidas PWM:** usadas para control continuo como motores, servos, dimmers y sonido.

- **Salidas digitales:** usadas para switches ON/OFF, relés, luces y disparadores.
- **Salidas de eventos:** pulsos cortos activados por comandos de un solo disparo.

6.4 Mapa de pines

Función	GPIO	Descripción
Salidas PWM		
Servo Dirección	18	Control PWM de dirección
Motor Izquierdo	19	Velocidad del motor
Cámara Pan	17	Servo PWM auxiliar
Motor Derecho	16	Velocidad del motor
Dimmer LED	23	Brillo PWM
Buzzer	27	Salida de sonido PWM
Switches digitales		
Switch 1	32	Salida digital
Switch 2	33	Salida digital
Switch 3	25	Salida digital
Switch 4	26	Salida digital
Switch 5	4	Salida digital
Switch 6	5	Salida digital
Switch 7	12	Salida digital
Switch 8	13	Salida digital
Salidas de eventos		
Evento 1	14	Salida de pulso
Evento 2	15	Salida de pulso
Evento 3	2	LED integrado
Evento 4	0	Pin de arranque (cuidado)

6.5 Configuración PWM

Tipo de salida	Frecuencia	Resolución
Servo	50 Hz	16 bits
Motor	20 kHz	8 bits
LED	5 kHz	8 bits
Buzzer	4 kHz	8 bits

6.6 Función de inicialización

```
void controlInit() {  
    pinMode(PIN_SW1, OUTPUT);  
    ...  
    pinMode(PIN_EVT4, OUTPUT);  
  
    ledcAttach(PIN_STEER,    SERVO_FREQ, SERVO_RES);  
    ledcAttach(PIN_MOTOR_L,  MOTOR_FREQ, MOTOR_RES);  
    ledcAttach(PIN_PAN,      SERVO_FREQ, SERVO_RES);  
    ledcAttach(PIN_MOTOR_R,  MOTOR_FREQ, MOTOR_RES);  
    ledcAttach(PIN_LED,      LED_FREQ,   LED_RES);  
    ledcAttach(PIN_BUZZER,   BUZZ_FREQ,  BUZZ_RES);  
}
```

Esta función prepara cada GPIO y configura todos los canales PWM.

6.7 Bucle principal de control

6.7.1 Mapeo de sticks

```
uint32_t steerPWM = mapServo(rcStatePacket.data.leftStickX);  
uint32_t motorL   = mapServo(rcStatePacket.data.leftStickY);  
uint32_t motorR   = mapServo(rcStatePacket.data.rightStickY);  
uint32_t panPWM   = mapServo(rcStatePacket.data.rightStickX);
```

6.7.2 Mapeo de servo

```
map(v, 0, 4095, 3277, 6553);
```

Convierte valores del joystick en anchos de pulso seguros para servos.

6.8 Mapeo de knobs

```
uint32_t ledPWM   = map(leftKnob, 0, 4095, 0, 255);  
uint32_t buzzPWM  = map(rightKnob, 0, 4095, 0, 255);
```

6.9 Decodificación de switches

Cada bit del byte de switches se asigna a un pin digital.

```
digitalWrite(PIN_SW1, sw & (1 << 0));  
...  
digitalWrite(PIN_SW8, sw & (1 << 7));
```

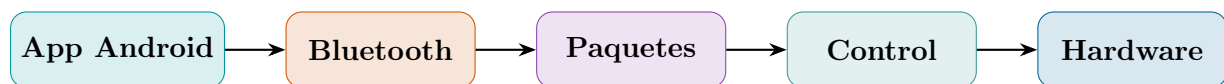
6.10 Manejo de eventos

```
void controlHandleEvent(byte eventId) {  
    switch (eventId) {  
        case 0x01: pulse(PIN_EVT1); break;  
        case 0x02: pulse(PIN_EVT2); break;  
        case 0x03: pulse(PIN_EVT3); break;  
        case 0x04: pulse(PIN_EVT4); break;  
        default: break;  
    }  
}
```

Cada evento genera un pulso corto de hardware.

6.11 Rol del sistema

La capa de control completa la cadena de señal RC:



7 Capa de Entrada (Telemetría Local de Sensores y Botones)

7.1 Propósito

La capa de entrada permite que el ESP32 no solo actúe como receptor de comandos, sino también como una **fente de datos** para la aplicación Android.

Este módulo:

- Lee sensores analógicos locales
- Lee botones y switches digitales
- Empaqueta los datos en una trama binaria de telemetría
- Envía los datos por Bluetooth hacia Android

Forma un canal de comunicación bidireccional donde tanto el teléfono como el ESP32 pueden generar telemetría en tiempo real.

7.2 Archivo de cabecera: `input.h`

```

#ifndef INPUT_H
#define INPUT_H
#include <Arduino.h>

void inputInit();
void inputUpdate();

#endif

```

7.3 Entradas soportadas

7.3.1 Entradas analógicas

El ESP32 soporta canales ADC en pines de solo entrada:

GPIO	Canal ADC	Uso típico
Entradas ADC		
34	ADC1_6	Potenciómetro, sensor
35	ADC1_7	Potenciómetro, sensor
36	ADC1_0	Divisor de batería
39	ADC1_3	Presión, temperatura

7.3.2 Entradas digitales

GPIO	Uso
Botones locales	
0	Botón / selección de modo
16	Botón
17	Botón

Todos los pines digitales usan el modo `INPUT_PULLUP` y por lo tanto leen LOW cuando están presionados.

7.4 Formato del paquete

La telemetría de entrada se empaqueta en la siguiente estructura binaria:

```

typedef struct {
    byte h1;        // 0xCC
    byte h2;        // 0x55
    uint16_t a34;
    uint16_t a35;
}

```

```

uint16_t a36;
uint16_t a39;
byte d0;
byte d16;
byte d17;
byte checksum;
} InputPacket;

```

7.5 Control de tiempo

El paquete se envía cada:

```
const unsigned long inputInterval = 50; // ms
```

Esto crea un flujo de telemetría de 20 Hz.

7.6 Inicialización

```

void inputInit() {
    pinMode(PIN_D0, INPUT_PULLUP);
    pinMode(PIN_D16, INPUT_PULLUP);
    pinMode(PIN_D17, INPUT_PULLUP);

    pinMode(PIN_A34, INPUT);
    pinMode(PIN_A35, INPUT);
    pinMode(PIN_A36, INPUT);
    pinMode(PIN_A39, INPUT);
}

```

7.7 Bucle principal de actualización

```

void inputUpdate() {
    if (!SerialBT.hasClient()) return;
    if (millis() - lastSend < inputInterval) return;
    lastSend = millis();
}

```

Esto asegura:

- No hay transmisión cuando no hay un dispositivo Android conectado
- Uso de ancho de banda fijo y controlado

7.8 Adquisición de datos

```

inputPacket.a34 = analogRead(PIN_A34);
inputPacket.a35 = analogRead(PIN_A35);
inputPacket.a36 = analogRead(PIN_A36);
inputPacket.a39 = analogRead(PIN_A39);

inputPacket.d0  = !digitalRead(PIN_D0);
inputPacket.d16 = !digitalRead(PIN_D16);
inputPacket.d17 = !digitalRead(PIN_D17);

```

7.9 Generación del checksum

```

byte c = 0;
for (int i = 2; i < INPUT_PACKET_SIZE - 1; i++)
    c += ((byte*)&inputPacket)[i];

inputPacket.checksum = c;

```

7.10 Transmisión

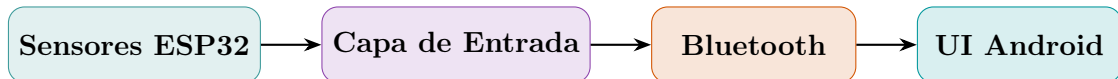
```

SerialBT.write((byte*)&inputPacket, INPUT_PACKET_SIZE);

```

7.11 Integración del sistema

La capa de entrada completa el bucle total de telemetría:



Esto permite que Android muestre:

- Voltaje de batería
- Datos de sensores externos
- Estados de botones físicos

8 Capa de Telemetría de Sensores I2C

8.1 Propósito

La capa de sensores I2C permite que el ESP32 lea sensores digitales externos y transmita sus valores en tiempo real a la aplicación Android utilizando el mismo canal de telemetría por Bluetooth.

Este módulo:

- Inicializa el bus I2C
- Lee sensores de temperatura
- Lee unidades de medición inercial (IMU)
- Empaqueta los valores de los sensores en una trama de telemetría
- Envía el paquete a la aplicación Android

El módulo es **solo de escritura**: nunca interpreta datos entrantes.

8.2 Interfaz pública: `i2c_sensors.h`

```
void i2cInit();
void sendI2CTelemetry();
```

8.3 Configuración del bus I2C

```
void i2cInit() {
    Wire.begin(21, 22);
}
```

- GPIO 21 → SDA
- GPIO 22 → SCL
- Se requieren resistencias pull-up estándar para I2C

8.4 Sensores soportados

8.4.1 Sensor de temperatura

- Dirección: 0x48
- Registro de temperatura de 16 bits

```
uint16_t readTemp() {
    Wire.beginTransmission(I2C_ADDR_TEMP);
    Wire.write(0);
    Wire.endTransmission();
    Wire.requestFrom(I2C_ADDR_TEMP, 2);
    return (Wire.read() << 8) | Wire.read();
}
```

8.4.2 IMU (acelerómetro)

- Dirección: 0x68
- Registros: 0x3B-0x40 (AX, AY, AZ)

```
void readIMU(int16_t &ax,int16_t &ay,int16_t &az) {  
    Wire.beginTransaction(I2C_ADDR_IMU);  
    Wire.write(0x3B);  
    Wire.endTransmission(false);  
    Wire.requestFrom(I2C_ADDR_IMU, 6);  
    ax = (Wire.read()<<8)|Wire.read();  
    ay = (Wire.read()<<8)|Wire.read();  
    az = (Wire.read()<<8)|Wire.read();  
}
```

8.5 Paquete de telemetría I2C

```
typedef struct {  
    byte h1;        // 0xCC  
    byte h2;        // 0x66  
    uint16_t temp;  
    uint16_t ax;  
    uint16_t ay;  
    uint16_t az;  
    byte sensorFlags;  
    byte checksum;  
} I2CPacket;
```

8.6 Frecuencia de transmisión

```
const unsigned long i2cInterval = 100; // ms
```

Esto genera un flujo de telemetría de 10 Hz.

8.7 Emisor de telemetría

```
void sendI2CTelemetry() {  
    if(!SerialBT.hasClient()) return;  
    if(millis()-lastSend<i2cInterval) return;  
    lastSend=millis();  
}
```

8.8 Construcción del paquete

```

i2cPacket.h1=0xCC;
i2cPacket.h2=0x66;
i2cPacket.temp = readTemp();
readIMU(ax, ay, az);

i2cPacket.ax=ax;
i2cPacket.ay=ay;
i2cPacket.az=az;
i2cPacket.sensorFlags=0b00000011;

```

8.9 Checksum y transmisión

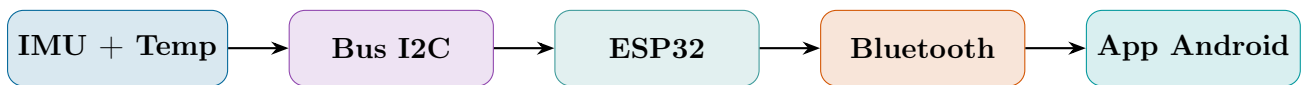
```

byte c=0;
for(int i=2;i<I2C_PACKET_SIZE-1;i++)
c+=((byte*)&i2cPacket)[i];

i2cPacket.checksum=c;
SerialBT.write((byte*)&i2cPacket,I2C_PACKET_SIZE);

```

8.10 Integración del sistema



Esto permite mostrar en tiempo real:

- Temperatura de la placa
- Aceleración del vehículo
- Detección de movimiento

9 Capa de Depuración y Diagnóstico

9.1 Propósito

La Capa de Depuración centraliza toda la salida formateada por `Serial` que se utiliza para monitorear, probar y validar el sistema de comunicación.

Este módulo:

- Decodifica campos binarios de los paquetes en texto legible
- Imprime los valores en un formato claro y fácil de interpretar
- Mantiene la lógica de depuración separada de la lógica de control

Este módulo **nunca** se comunica por Bluetooth y **nunca** modifica los datos. Solo lee las estructuras globales de paquetes.

9.2 Archivo de Cabecera: `debug.h`

```
#ifndef DEBUG_H
#define DEBUG_H

#include <Arduino.h>
#include "packets.h"

void printStatePacket();
void printEventPacket();
void printPanelPacket();
void printIndicatorPacket();
void printPlotPacket();

#endif
```

9.3 Archivo de Implementación: `debug.cpp`

9.3.1 Impresor de Paquetes de Estado

```
void printStatePacket() {
    Serial.println("\n--- RC STATE PACKET ---");

    Serial.printf("L Stick: X=%u Y=%u\n",
        rcStatePacket.data.leftStickX,
        rcStatePacket.data.leftStickY);

    Serial.printf("R Stick: X=%u Y=%u\n",
        rcStatePacket.data.rightStickX,
        rcStatePacket.data.rightStickY);

    Serial.printf("Knobs: L=%u R=%u\n",
        rcStatePacket.data.leftKnob,
        rcStatePacket.data.rightKnob);

    byte s = rcStatePacket.data.switches;
    Serial.printf("Switch Byte: 0x%02X\n", s);

    for (int i = 0; i < 6; i++) {
        Serial.printf(
            "  S%d = %s\n",
            i + 1,
            (s & (1 << i)) ? "ON" : "OFF"
        );
    }
}
```

```

Serial.printf("Checksum: %u\n",
rcStatePacket.data.checksum);

Serial.println("-----");
}

```

9.3.2 Impresor de Paquetes de Evento

```

void printEventPacket() {
Serial.println("\n--- EVENT PACKET ---");
Serial.printf("Event ID: 0x%02X\n",
rcEventPacket.data.eventId);
Serial.printf("Checksum: 0x%02X\n",
rcEventPacket.data.checksum);
Serial.println("-----");
}

```

9.3.3 Impresor de Telemetría del Panel

```

void printPanelPacket() {
Serial.println("\n--- PANEL TELEMETRY ---");
Serial.printf("Left: %u Right: %u\n",
panelPacket.leftPanelValue,
panelPacket.rightPanelValue);
Serial.printf("States: 0x%02X\n",
panelPacket.panelStates);
Serial.printf("Checksum: %u\n",
panelPacket.checksum);
}

```

9.3.4 Impresor de Telemetría de Indicadores

```

void printIndicatorPacket() {
Serial.println("\n--- INDICATOR TELEMETRY ---");
Serial.printf("Analog: %u\n",
indicatorPacket.analogValue);
Serial.printf("Battery: %u%%\n",
indicatorPacket.batteryLevel);
Serial.printf("Checksum: %u\n",
indicatorPacket.checksum);
}

```

9.3.5 Impresor de Telemetría de Gráficas

```

void printPlotPacket() {
Serial.println("\n--- PLOT TELEMETRY ---");
Serial.printf("Plots: %u\n",

```

```
plotPacket.numPlots);  
Serial.printf("P1=%u P2=%u P3=%u\n",  
plotPacket.plotValue1,  
plotPacket.plotValue2,  
plotPacket.plotValue3);  
Serial.printf("Checksum: %u\n",  
plotPacket.checksum);  
}
```

9.4 ¿Por qué una Capa de Depuración Dedicada?

- Mantiene el código de control y telemetría limpio
- Permite habilitar o deshabilitar fácilmente los mensajes por Serial
- Evita problemas de temporización en la lógica Bluetooth
- Ayuda a visualizar estructuras de datos binarias